Uniwersytet Gdański
Wydział Matematyki, Fizyki i Informatyki
Instytut Informatyki

Nonogram za pomocą algorytmów genetycznych

Maria Koren

Gdańsk

9 czerwca 2024

Spis treści

1 Wstęp **2**

2 Algorytm **3**

3 Porówanie **5**

4 Krzywy uczenia się **6**

5 Aplikacja **8**

6 Podsumowanie **9**

1 Wstęp

W tym projekcie jest przedstawiony problem nonogram.

Nonogram (malowanie liczbami) to łamigłówka polegająca na zaczernianiu pól diagramu. Zaczernione pola utworzą rysunek. To, które pola trzeba zaczernić, wskazują liczby obok diagramu. Liczby z lewej strony każdego wiersza określają, ile grup czarnych pól jest w danym rzędzie i ile czarnych pól jest w każdej grupie.Analogicznie jest z liczbami u góry diagramu.

Powyższy problem rozwiązano za pomocą algorytmu genetycznego (użyto 2 fukcje fitness) dla trzech rozmiarów: 5x5, 10x10, 15x15.

Dodatkowo napisana mała aplikacja dla nonogramu 5x5.

2 Algorytm

Dla powyższego problemu wybrano geny zero-jedynkowe, gdzie 1 oznacza zamalowaną klatkę, 0 pustą.

Użyto funkcji pomocniczej dla funkcji fitness:

```
def fitness_sequence(sequence, clues):
        fitness = 0
        seq_id = 0

        for clue in clues:
            #  Skip zeros in sequence
            while seq_id < len(sequence) and sequence[seq_id] == 0:
                seq_id += 1

            if seq_id == len(sequence):
                break

            # Count ones and check it matches to clue
            block_len = 0
            while seq_id < len(sequence) and sequence[seq_id] == 1:
                block_len += 1
                seq_id += 1
            if block_len == clue:
                # +1 point for each correct block
                fitness += 1

        if(seq_id < len(sequence)):
            # -5 point for each additional block
            fitness -= np.sum(sequence[seq_id:])*5

        return fitness
```

Ta funkcja zlicza poprawne wartości i daje +1 punkt za każdą poprawną zgadniętą wartość i karę -5 za każdą niepoprawną.

Główna funkcja *fitness*:

```
def fitness_func(model, solution, solution_idx):
        # Splits nonogram for columns and rows
        col_clues = np.array(nonogram[0], dtype=object)
        row_clues = np.array(nonogram[1], dtype=object)

        # Splits soluton to arrays of nonogram size
        grid = solution.reshape((len(row_clues), len(col_clues)))
        score = 0

        #  Checks each column solution
        for i in range(len(col_clues)):
            col_score = fitness_sequence(grid[:, i], col_clues[i])
            score += col_score

        # Checks each row solution
        for j in range(len(row_clues)):
            row_score = fitness_sequence(grid[j, :], row_clues[j])
            score += row_score
        return score
```

Główna funkcja fitness wywołuje pomicniczą dla każdej kolumny i każdego wiersza i zlicza sumarną wartość fitness (score).

Zrobiono również program z inną funkcją *fitness*, która od poprzedniej różni się karą: tym razem za niepoprawne zgadywanie kara wynosi -1 pkt.

3 Porówanie

Żeby porównać wyniki funkcji, użyto po 3 przykłady na każdy rozmiar. Wyniki są reprezentowane w tabeli 1.

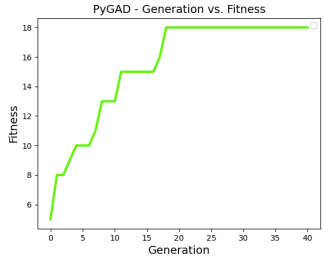
Rozmiar	Algorytm	Statystyka	Wartości	Średnia
5x5	nonogram_ga	Accuracy	100.0, 100.0, 83.33	94.44
		Time	1.91, 1.60, 3.47	2.33
	nonogram_ga2	Accuracy	100.0, 100.0, 83.33	94.44
		Time	10.33, 1.82, 3.18	5.11
10x10	nonogram_ga	Accuracy	70.37, 82.98, 92.68	82.01
		Time	23.98, 24.58, 26.17	24.91
	nonogram_ga2	Accuracy	77.78, 87.23, 85.37	83.46
		Time	27.38, 32.27, 33.64	31.10
15x15	nonogram_ga	Accuracy	55.84, 70.59, 43.1	56.51
		Time	49.84, 102.65, 50.30	67.60
	nonogram_ga2	Accuracy	66.23, 78.82, 50.0	65.02
		Time	55.45, 111.30, 52.62	73.12

Tabela 1: Wyniki dla różnych algorytmów i rozmiarów nonogramów

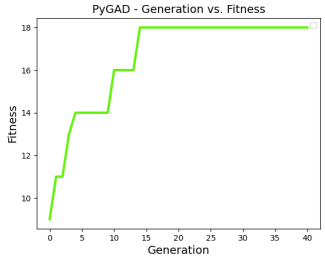
Na podstawie przedstawionych wyników można stwierdzić, że algorytm 1 zawsze działa szybciej niż algorytm 2, i daje nie gorsze wyniki.

4 Krzywy uczenia się

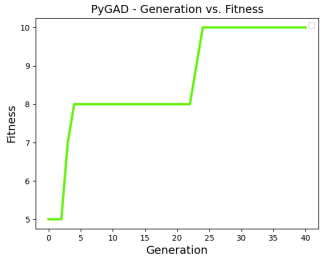
Dla fittnes 1:



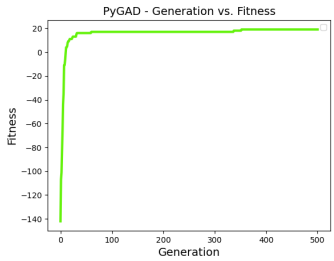
Rysunek 1: 5x5, przykład 1



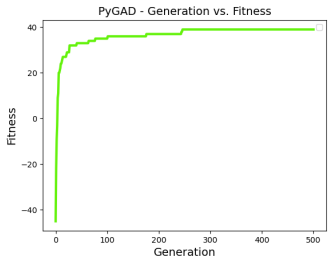
Rysunek 2: 5x5, przykład 2



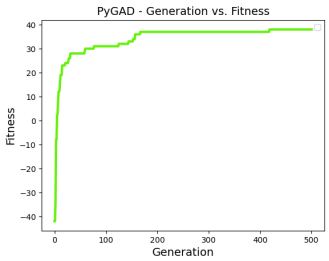
Rysunek 3: 5x5, przykład 3



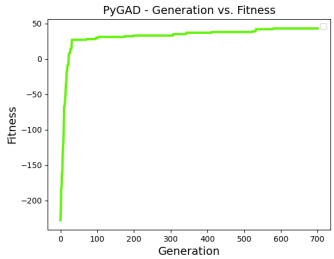
Rysunek 4: 10x10, przykład 1



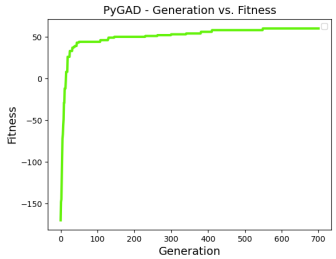
Rysunek 5: 10x10, przykład 2



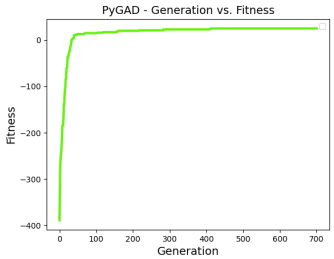
Rysunek 6: 10x10, przykład 3



Rysunek 7: 15x15, przykład 1

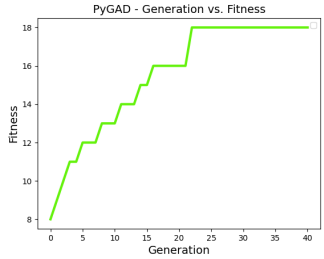


Rysunek 8: 15x15, przykład 2

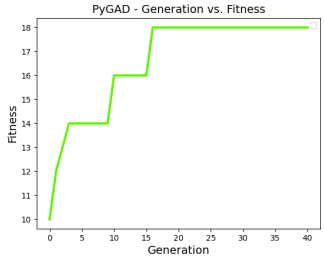


Rysunek 9: 15x15, przykład 3

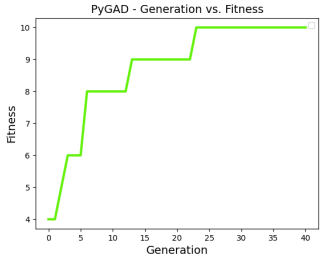
Dla fitness 2:



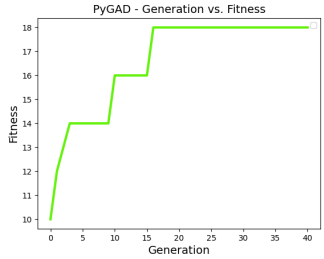
Rysunek 10: 5x5, przykład 1



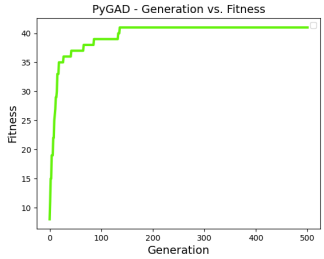
Rysunek 11: 5x5, przykład 2



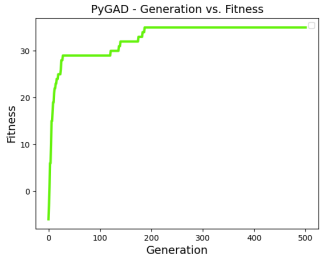
Rysunek 12: 5x5, przykład 3



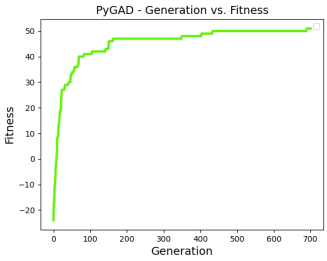
Rysunek 13: 10x10, przykład 1



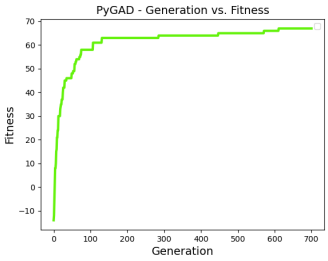
Rysunek 14: 10x10, przykład 2



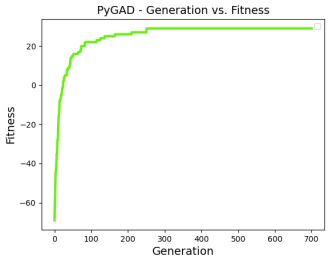
Rysunek 15: 10x10, przykład 3



Rysunek 16: 15x15, przykład 1



Rysunek 17: 15x15, przykład 2



Rysunek 18: 15x15, przykład 3

5 Aplikacja

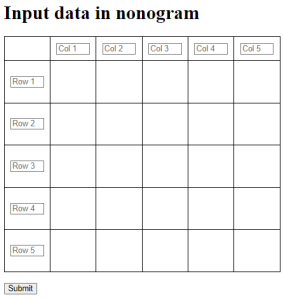
Przedstawione zostały wyniki dla 3 nonogramów każdego rozmiaru, a przetestowano było więcej. Zauważono, że dla nonogramu 5x5 wyniki często wychodzą 100%. Oraz wyniki są otrzymywane dość szybko. Dlatego postanowiono napisać aplikacje właśnie dla tego rozmiaru.

Dla aplikcji wzięto algorytm z pierwszą funkcją fitness, ponieważ dokładność jest ta sama, natomiast jest szybsza, co w sytuacji aplikacji ma znaczenie.

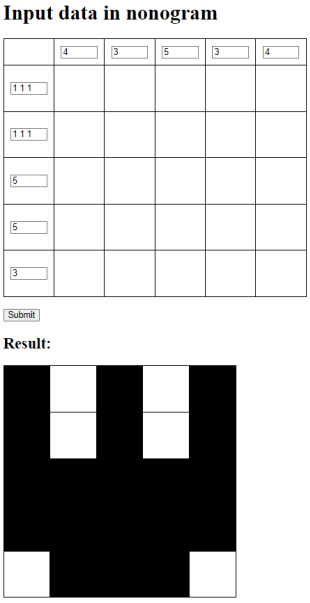
W celu uruchamienia aplikcji trzeba wejść do folderu server i użyć polecenia:

```
python main.py
```

Zatem wejśc w przeglądarkę pod strone *localhost:5000* i będzie widoczna strona:



Podając liczby, opisujący chcące zamalowanie i naciskając przycisk **submit** otrzymujemy niżej wyniki. Przykład poniżej:



6 Podsumowanie

Zostały napisane 2 programy używające algorytmy genetyczne dla rozwiązania problemu nonogram. Porównując algorytmy, wyszło, że jeden algorytm działa szybciej, ale daje gorsze wyniki. Drugi na odwrót działa wolniej, ale daje lepsze wyniki.

Ponadto dla problemu w rozmiarze 5x5 napisano aplikacje, która używa ten algorytm rozwiązywania nonogramu.